Shondhan

AI-powered missing person finder

Project Overview: Login and Family Signup Module

Everything built so far, how the parts connect, how to run it, problems solved along the way, and what comes next.

Document date	12 November 2026
Stack	Django + Django REST Framework + SimpleJWT, PostgreSQL, React + Vite (static build)
Status	Backend verified running with PostgreSQL by the developer; signup added afterwards and not yet tested end to end
Deliverable	shondhan-login.zip (backend/ and frontend/)

1. Where the project stands

Shondhan is a web app that helps families find missing people by matching family reports against records posted by police stations and hospitals. The full vision (SRS, matching engine, reporter and officer screens) is large. In this phase we deliberately built only the foundation: **authentication**, so that every later feature has real users to hang on.

What was built in this phase

- A Django REST backend with a custom user model, JWT login, family registration and a protected 'who am I' endpoint.
- A PostgreSQL database configured through environment variables.
- A React (Vite) frontend with a bilingual login page, a family signup wizard and a placeholder dashboard.
- Dark and light themes, a Bangla/English toggle and a glass-style interface using the project colour palette.
- Local setup instructions, including PostgreSQL installation on Ubuntu, and fixes for the problems met while running it.

What is intentionally not built yet

- Staff screens (police, medical, case officer, admin, auditor), missing-person case forms, photo upload, AI matching, merging, notifications, deceased-record handling and the audit ledger.
- These are in the SRS and are the next phases (see section 11).

2. How the three parts connect

```
Browser (React, static files)
    |   HTTP + JSON      Authorization: Bearer <access token>
    v
Django REST API  (localhost:8000)      /api/auth/...
    |   SQL (psycopg 3)
    v
PostgreSQL       (localhost:5432)      database: shondhan
```

Link	How it is wired	Where to look
React to Django (development)	Vite dev server (port 5173) forwards every /api/* request to Django on port 8000. No CORS needed locally.	frontend/vite.config.js
React to Django (production)	The built site calls the API origin set in VITE_API_URL. Django allows that origin through CORS_ALLOWED_ORIGINS.	frontend/.env, backend/.env
Django to PostgreSQL	settings.py reads DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT from backend/.env.	backend/config/settings.py
Browser identity	Login returns a JWT access token. The frontend keeps it in sessionStorage and sends it as a Bearer header.	frontend/src/api.js

3. Project structure

```
shondhan-login/
  backend/
    manage.py
    requirements.txt
    .env.example              -> copy to .env
    config/
      settings.py             database, JWT, CORS, custom user
      urls.py                 /admin/ and /api/auth/
      wsgi.py
    accounts/
      models.py               custom User
      serializers.py          UserSerializer, RegisterSerializer
      views.py                LoginView, RegisterView, MeView
      urls.py                 login/ register/ refresh/ me/
      admin.py                user admin with extra fields
  frontend/
    package.json  vite.config.js  index.html  .env.example
    src/
      main.jsx  App.jsx  api.js  prefs.jsx  i18n.js  styles.css
      pages/  Login.jsx  Signup.jsx  Dashboard.jsx  Toggles.jsx
```

4. Backend in detail

4.1 Technology

Package	Purpose
Django 5.x	Web framework, admin site, ORM, migrations
djangorestframework	REST API views and serializers
djangorestframework-simplejwt	JWT access and refresh tokens
django-cors-headers	Cross-origin access for the production frontend
psycopg[binary] 3	PostgreSQL driver
python-dotenv	Loads backend/.env

4.2 Custom user model (accounts.User)

Extends Django's AbstractUser. A custom user model had to be chosen before the first migration, because changing it later is painful.

Field	Type	Meaning
username	text (built in)	Set to the phone number at signup
first_name	text (built in)	Stores the full name; exposed to the API as full_name
email	text (built in)	Optional
phone	text, unique, nullable	Primary login identifier, format 01XXXXXXXXX
role	choice	family, police, medical, officer, admin, auditor (default family)
failed_attempts	small integer	Wrong-password counter for lockout
locked_until	datetime, nullable	Set when the account is locked
age	small integer	Added with signup; must be 18 or older
relationship	text	Added with signup (see note in section 6)
consent_version / consent_at	text / datetime	Records which consent text was accepted and when

4.3 Login logic

- The user types a phone number or an email plus a password. The server finds the user by either.
- If the account is locked, it answers 429 with code 'locked'.
- Wrong user, wrong password or an inactive account all return the same 401 'invalid' answer, so an attacker cannot tell which accounts exist.
- Each wrong password adds one to failed_attempts. At 5, the account is locked for 15 minutes and the counter resets.
- On success the counters reset and the API returns an access token (15 minutes), a refresh token (7 days) and the user profile.
- Staff accounts are created inactive by an admin; being inactive blocks login, which implements 'staff must be approved'.

4.4 Registration logic (family only)

- Fields: full name, phone, optional email, age, relationship, password, consent.
- Phone must be 11 digits starting with 01. Bangla digits are converted to normal digits on the server. Duplicate phones and emails are rejected.
- Age must be 18 or older (the SRS says minors cannot create cases).
- Password goes through Django's validators: minimum 8 characters, not too common.
- Consent must be true; the consent version ('v1') and time are stored.
- Every self-registered user gets role 'family'. Staff roles can only be set in the admin site.

4.5 API reference

Method and path	Auth	Request	Response
POST /api/auth/login/	none	identifier, password	200: access, refresh, user. 401: code invalid. 429: code locked
POST /api/auth/register/	none	full_name, phone, email?, age, relationship, password, consent	201: ok. 400: field errors
GET /api/auth/me/	Bearer	-	Current user profile
POST /api/auth/refresh/	none	refresh	New access token
GET /admin/	session	-	Django admin site (users, roles, unlock accounts)

5. Frontend in detail

5.1 Technology and pages

Piece	Role
React 18 + Vite 5	Build tooling; npm run build produces a static dist/ folder
react-router-dom 6	Multiple real pages, not a single page
No UI library	Plain CSS with variables, so the glass look is easy to control

Route	Page	Notes
/login	Login.jsx	Phone/email + password, show/hide password, bilingual errors, link to signup
/signup	Signup.jsx	Two-step wizard: (1) name, phone, email, age, relationship; (2) password, repeat password, consent
/dashboard	Dashboard.jsx	Protected. Calls /me/, greets the user, has a logout button. Placeholder for later features
any other path	-	Redirects to /login

5.2 Key files

File	Responsibility
api.js	One fetch wrapper. Maps server answers to simple error codes (invalid, locked, validation, network). Stores the token in sessionStorage.
prefs.jsx	React context holding language (default Bangla) and theme (default dark). Saved in localStorage and applied to the html element.
i18n.js	All text in Bangla and English, including every error message and dropdown label.
styles.css	Colour variables for both themes, glass panels (blur and translucency), 44 px touch targets, visible focus rings, reduced-motion support, mobile layout.
Toggles.jsx	Language and theme buttons shown at the top of every page.

5.3 Design rules applied

- Bangla first, English toggle, everything bilingual.
- Dark mode default with palette #2A1A0E, #D4621A, #F4AE52, #FEF3E2. Light mode with the cream and sage palette.
- Glass panels with transparent backgrounds, one concern per signup step, large inputs and buttons, easy wording.
- Hind Siliguri web font so Bangla text renders cleanly.

6. Family signup: what was added and why

The signup follows the family scenario in your brief and SRS: a family member gives their name, phone, optional email, age and relationship, and accepts consent.

- **Backend:** four new user fields (age, relationship, consent_version, consent_at) and a stricter RegisterSerializer. A new migration is required.
- **Frontend:** new Signup page, new API call, new route, a signup link on the login page, and the new Bangla/English messages.
- **Auto login:** after a successful signup the app logs the user in and opens the dashboard.
- **Design note:** in the SRS the relationship belongs to each missing-person case. It is stored on the profile for now, and the case form will ask for it again later.

7. Database setup (Ubuntu)

```
sudo apt update
sudo apt install postgresql postgresql-contrib libpq-dev
sudo systemctl enable --now postgresql
sudo -u postgres psql
    CREATE USER shondhan WITH PASSWORD 'shondhan_pass';
    CREATE DATABASE shondhan OWNER shondhan;
    \q
psql -h localhost -U shondhan -d shondhan      # connection test
```

The developer confirmed this connection works. 'shondhan_pass' is a development password only and must be changed for any online deployment.

Inspecting tables

Method	Command or action
psql	\dt (list tables), \d accounts_user (columns), SELECT id, username, email, phone, role FROM accounts_user;
Django	python manage.py dbshell
Admin site	http://127.0.0.1:8000/admin/ then Accounts, Users
GUI	DBeaver or pgAdmin (host localhost, port 5432, db shondhan)

Expected tables after migrate: accounts_user plus Django's auth_group, auth_permission, django_admin_log, django_content_type, django_migrations and django_session.

8. Running the project, step by step

Backend (terminal 1)

```
cd backend
python -m venv venv
source venv/bin/activate           # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # Windows: copy .env.example .env
# edit .env so DB_PASSWORD matches PostgreSQL
python manage.py makemigrations accounts
python manage.py migrate
python manage.py createsuperuser   # give username, EMAIL and password
python manage.py runserver
```

Frontend (terminal 2)

```
cd frontend
npm install
npm run dev
# open http://localhost:5173
```

Checks

- http://127.0.0.1:8000/admin/ shows the Django login: backend and database are connected.
- http://127.0.0.1:8000/api/auth/me/ returns a 401 JSON message: the API is alive.
- http://localhost:5173/signup, create a family account, and you land on the dashboard.
- Verify the row: psql -h localhost -U shondhan -d shondhan -c "SELECT username, first_name, phone, age, relationship, role FROM accounts_user;"

Updating an existing copy after the signup change

Replace backend/accounts/ and frontend/src/ with the new versions (keep your .env and venv), then run makemigrations accounts and migrate again.

9. Problems met and how they were solved

Problem	Cause	Fix
'copy: command not found' on Ubuntu	copy is a Windows command	Use cp .env.example .env
Did not know how to set up PostgreSQL	Fresh machine	Installed with apt, created role and database in psql (section 7)
Browser showed 404 at http://127.0.0.1:8000/	The backend is an API with no home page; only /admin/ and /api/auth/ exist	Not an error. Open /admin/ or run the React frontend. The favicon 404 is harmless
Login could fail for a new superuser	Login uses phone or email, and a superuser created with only a username has neither	Give an email at createsuperuser, or set the phone in the admin site
Locked out after 5 wrong tries	Intentional security rule	Wait 15 minutes or clear locked_until in the admin site
Possible 'permission denied for schema public' on PostgreSQL 15+	Schema permissions	Not needed here because shondhan owns the database; otherwise GRANT ALL ON SCHEMA public

10. Security notes and production deployment

Already in place

- 15-minute access tokens; lockout after 5 failed logins for 15 minutes.
- Identical error for wrong user and wrong password.
- Password hashing and validators from Django; minimum length 8.
- Under-18 users cannot register.
- Versioned consent stored with a timestamp.
- Role set server-side; users cannot choose a staff role.
- Secrets and database settings come from .env, which is git-ignored.

Before going online

- Set DJANGO_DEBUG=0, a long random DJANGO_SECRET_KEY, and DJANGO_ALLOWED_HOSTS.
- Serve Django with gunicorn behind HTTPS; change the database password.
- Build the frontend (set VITE_API_URL, run npm run build) and host dist/ on Nginx or a static host with a fallback to index.html so refreshing /dashboard works.
- Set CORS_ALLOWED_ORIGINS to the real frontend address.
- Move the refresh token to an HttpOnly cookie; add automatic token refresh in the frontend.

11. Honest status and next steps

Verification status

- Verified by the developer: PostgreSQL installed and reachable, Django dev server starts with no system-check issues, admin and API URLs respond.
- Written but not yet confirmed by running: the login screen against a real account and the new signup flow. The first test is a signup at /signup followed by the SQL check above.

Known gaps in this module

- No forgot-password flow, email or phone verification, or staff 2FA.
- No automated tests yet for login lockout and registration.
- Frontend is JavaScript, not TypeScript as the SRS suggests.
- Sessions end when the browser tab closes because the token lives in sessionStorage.

Suggested build order

Phase	Work	Why this order
1	Test signup and login end to end; add backend tests for lockout and registration	Locks in the foundation
2	Role-aware dashboards, token refresh, profile page	Every later screen depends on roles
3	Missing-person case form for families (3 photos minimum, optional ID documents, 5 active cases limit)	First real feature; gives data to match against
4	Police and medical reporter forms with dropdowns and sub-dropdowns	Creates the found stream
5	Face embedding service (InsightFace via ONNX) and matching with percentage bands	Core value of the product
6	Notifications, duplicate merge workflow, case officer queue, resolution and deletion, deceased-record protections, audit ledger	Completes the SRS flows

A match is never a verdict: a human confirms every result. This principle from the SRS should be kept in every later phase.